

CONTENEDORES Y ORQUESTACIÓN



&



Estudiantes:

- 1. JOEL LEIVA CALLE**
- 2. JOSÉ DAVID CACEREZ RIVERO**
- 3. JOSUÉ DAVID COLQUE CANDIA**



INTRODUCCIÓN

El proyecto "Funciona" en la computadora de origen porque utiliza versiones específicas de herramientas (Node.js v10.1.1 y Python v3.2.0), pero falla ("No Funciona") en la de destino debido a que esta tiene instaladas versiones ligeramente diferentes (Node.js v10.1.2 y Python v3.2.1).

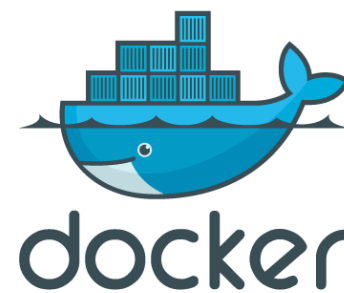
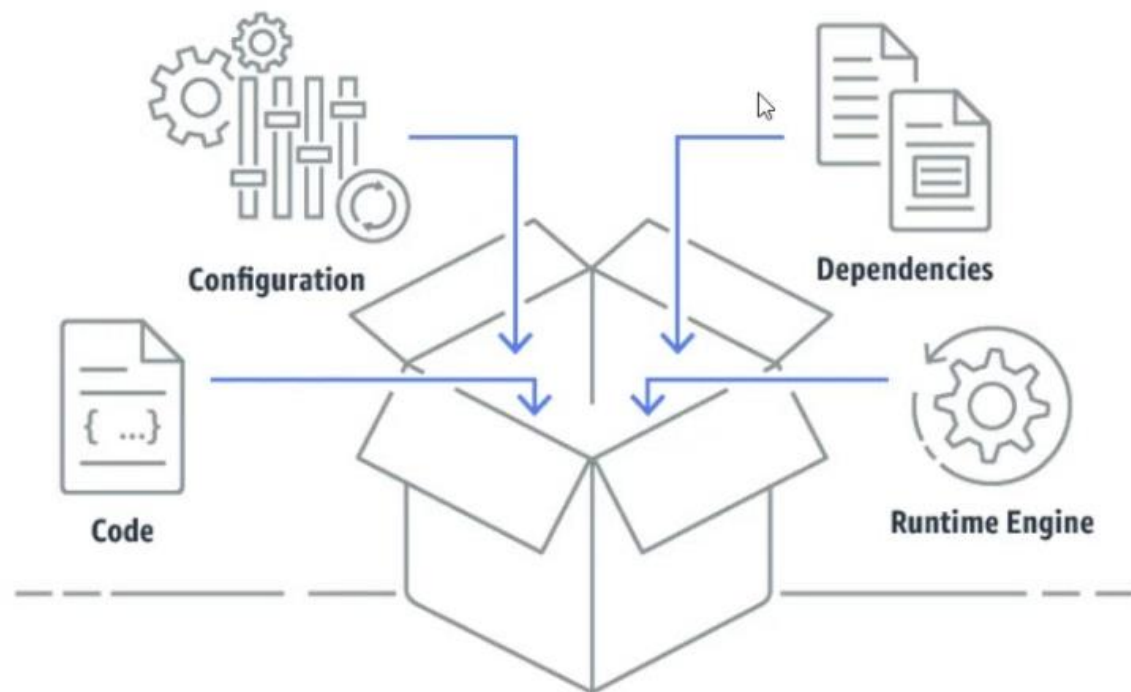


CONTENEDOR

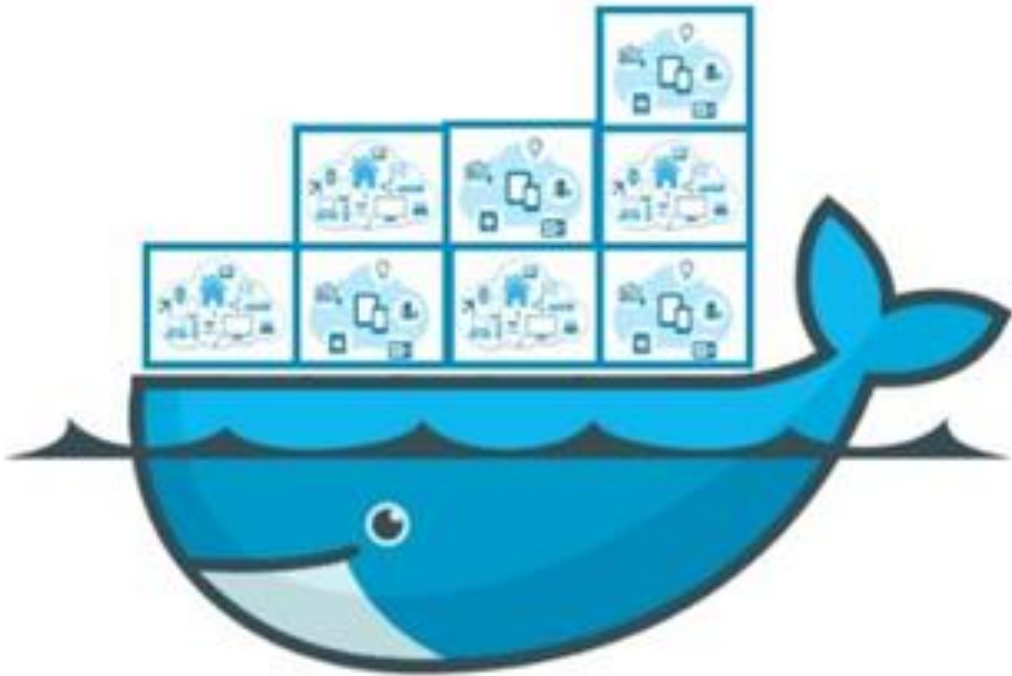
Un contenedor es una forma de empaquetar una aplicación junto con todo lo que necesita para funcionar (librerías, dependencias, configuración), de manera que pueda ejecutarse igual en cualquier computadora o servidor.

¿CÓMO CREAR UN CONTENEDOR?

Para la creación de contenedores existen diversas herramientas como Podman, containerd o LXC, pero, la más popular y ampliamente utilizada en la industria es Docker



DOCKER

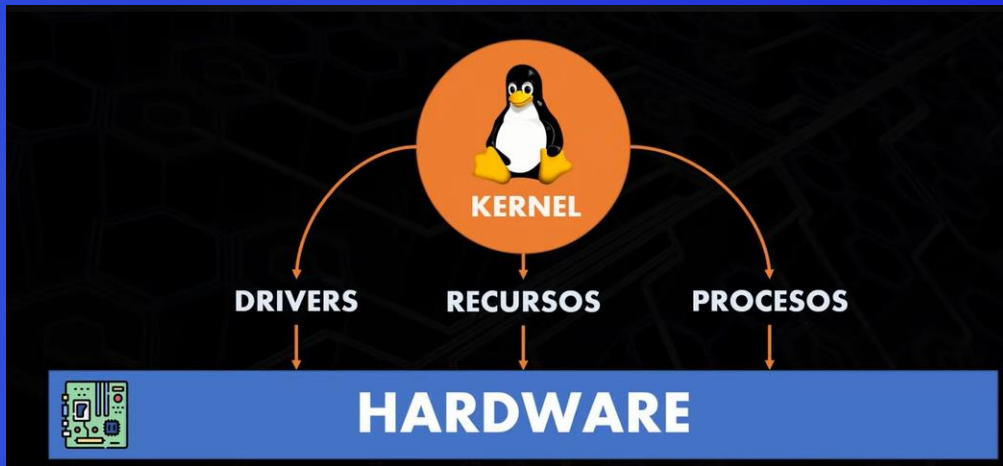


Docker es una herramienta (plataforma de software de código abierto) que nos permite crear, ejecutar, gestionar y distribuir contenedores, empaquetando aplicaciones junto con todas sus dependencias para que funcionen de forma aislada y en cualquier sistema.

Su objetivo principal es resolver el problema de "funciona en mi computadora, pero no en la tuya" al garantizar que el software se ejecute de la misma forma en cualquier entorno.

¿CÓMO FUNCIONA DOCKER?

Docker fue diseñado para aprovechar el kernel de Linux.



Un **kernel** es la parte central de un sistema operativo que actúa como intermediario entre el hardware (procesador, memoria, disco) y el software, gestionando los recursos del sistema y permitiendo que los programas funcionen correctamente.

Docker usa funciones del kernel de Linux que ya existen en él para crear contenedores eficientes:

- **Namespaces:** permiten el **aislamiento**. Hacen que cada contenedor “crea” que está ejecutándose en su propio sistema independiente, separando procesos, red, usuarios y sistema de archivos. Por ejemplo, un contenedor no puede ver ni afectar los procesos de otro.
- **cgroups (control groups):** permiten el **control de recursos**. Se encargan de limitar y distribuir el uso de CPU, memoria, disco y red para cada contenedor, evitando que uno solo consuma todos los recursos del sistema.

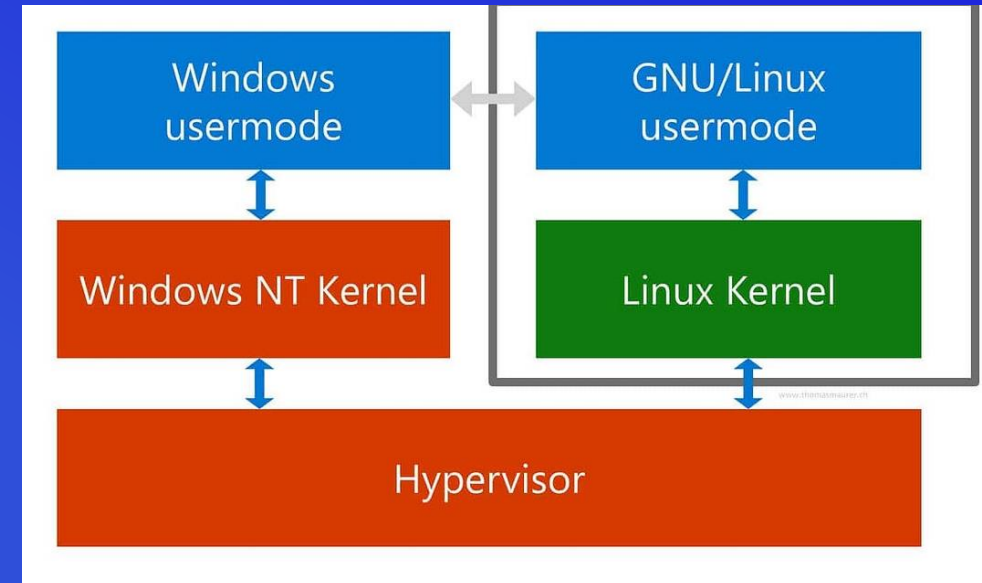
¿CÓMO INSTALAR DOCKER?

Docker en Windows



Tendríamos que tener Windows 10 o 11 porque Microsoft ha implementado una tecnología llamada **WSL (Subsistema de Windows para Linux)**, la cual permite ejecutar un kernel Linux real dentro de Windows. Gracias a esto podemos instalar una distribución de Linux que funciona como un “mini sistema independiente”, pero al mismo tiempo está integrada con Windows, permitiendo que ambos sistemas trabajen de forma fluida sin necesidad de una máquina virtual pesada tradicional. Y gracias a esa integración, podemos instalar Docker dentro de esa distribución de Linux y utilizar directamente el kernel de Linux, que es el que permite su funcionamiento correcto.

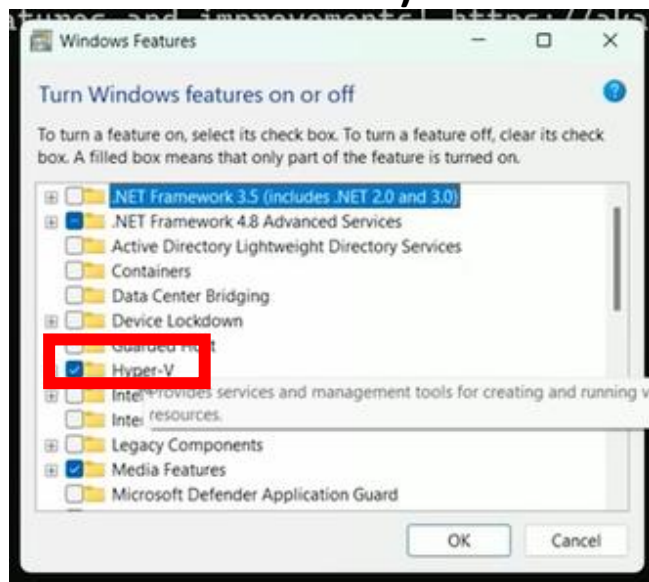
WSL



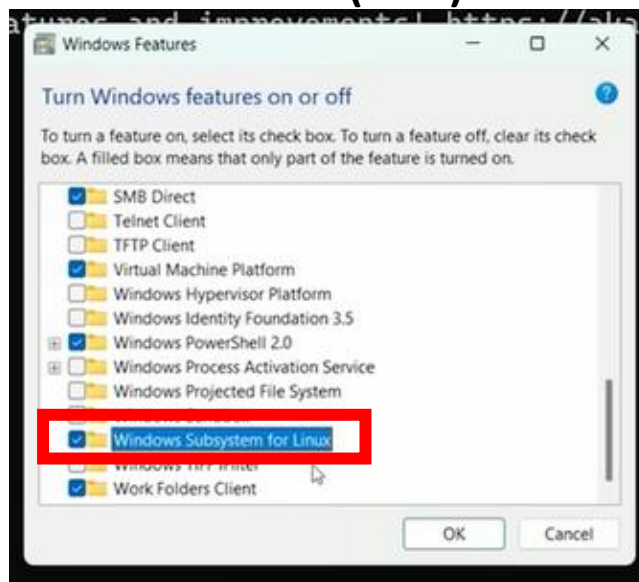
PROCESO DE INSTALACIÓN DE LINUX EN WINDOWS

Primero debemos asegurarnos de habilitar en “características de Windows” las opciones :

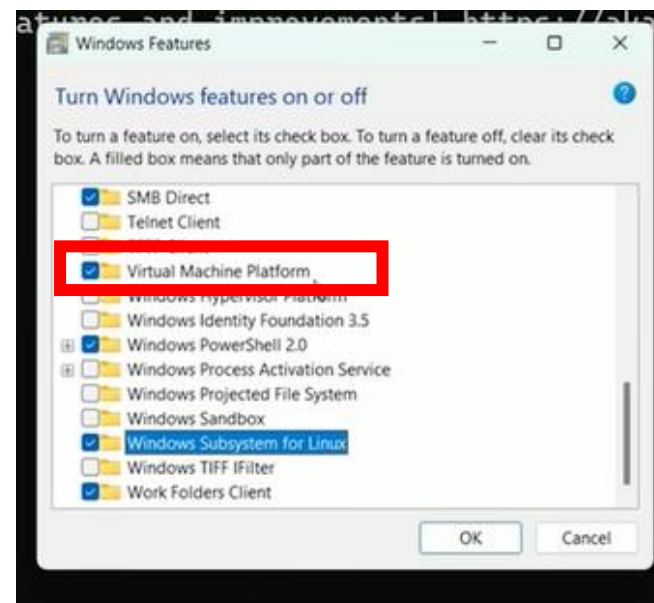
Hyper-V (Virtualización de Windows)



Subsistema de Windows para Linux (WSL)



Plataforma de máquina virtual (Virtual Machine Platform)



Reiniciamos el sistema

Con estas funciones permiten que Windows pueda crear y ejecutar entornos virtuales donde WSL instala un kernel Linux real.

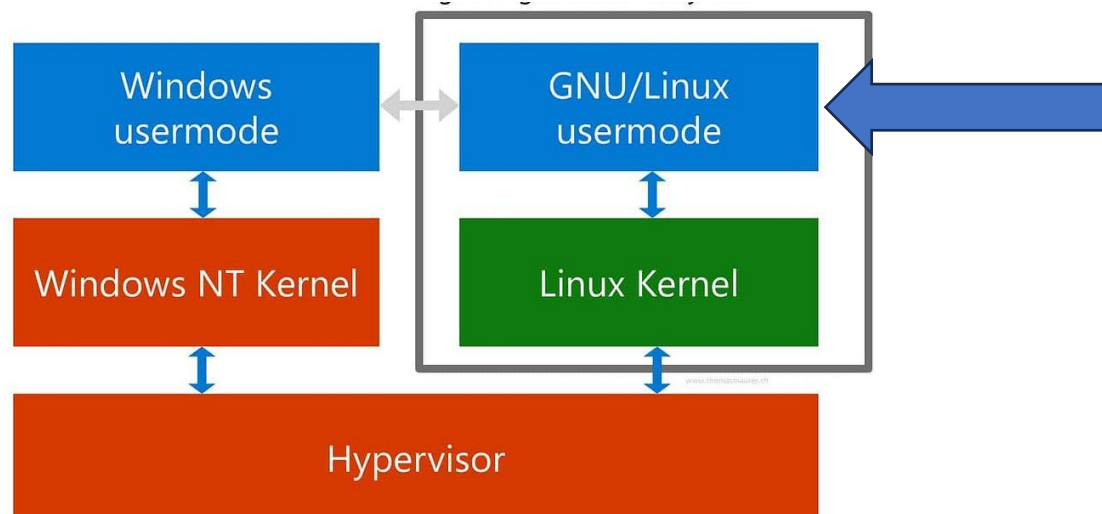
AHORA SÍ PODEMOS ACTIVAR E INSTALAR WSL CON LOS SIGUIENTES PASOS:

Abrimos **PowerShell como administrador** y ejecutamos:

```
wsl --install
```

Al ejecutar el comando, Windows realiza las siguientes tareas:

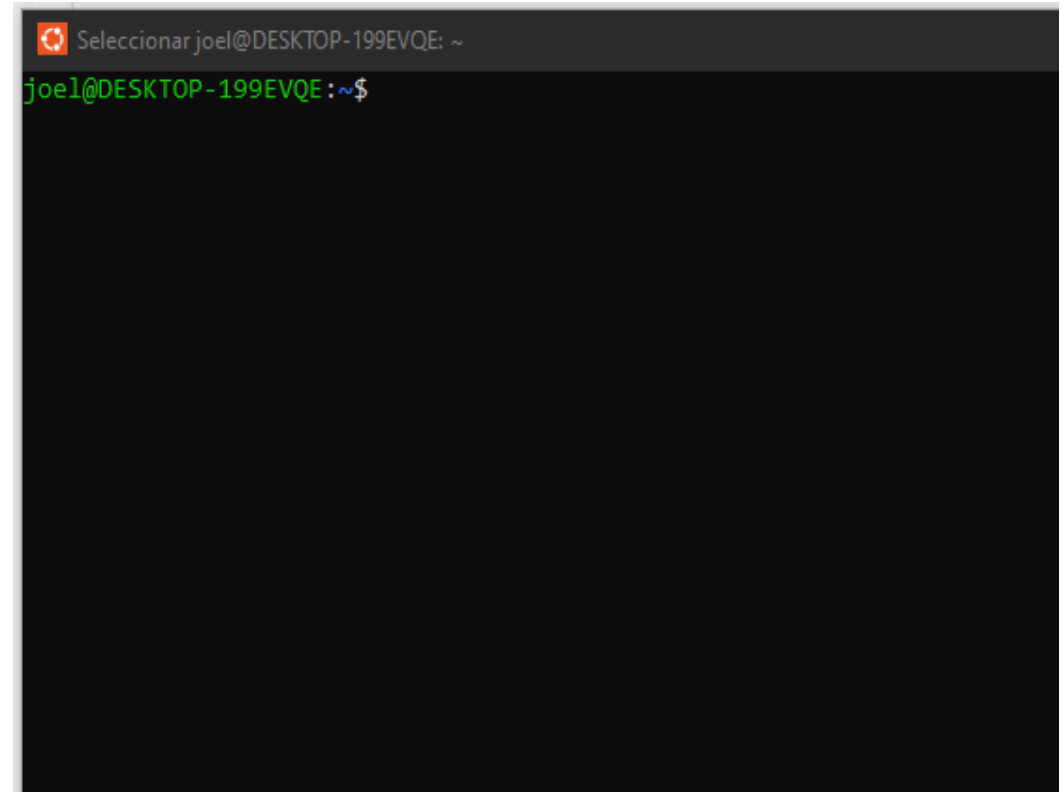
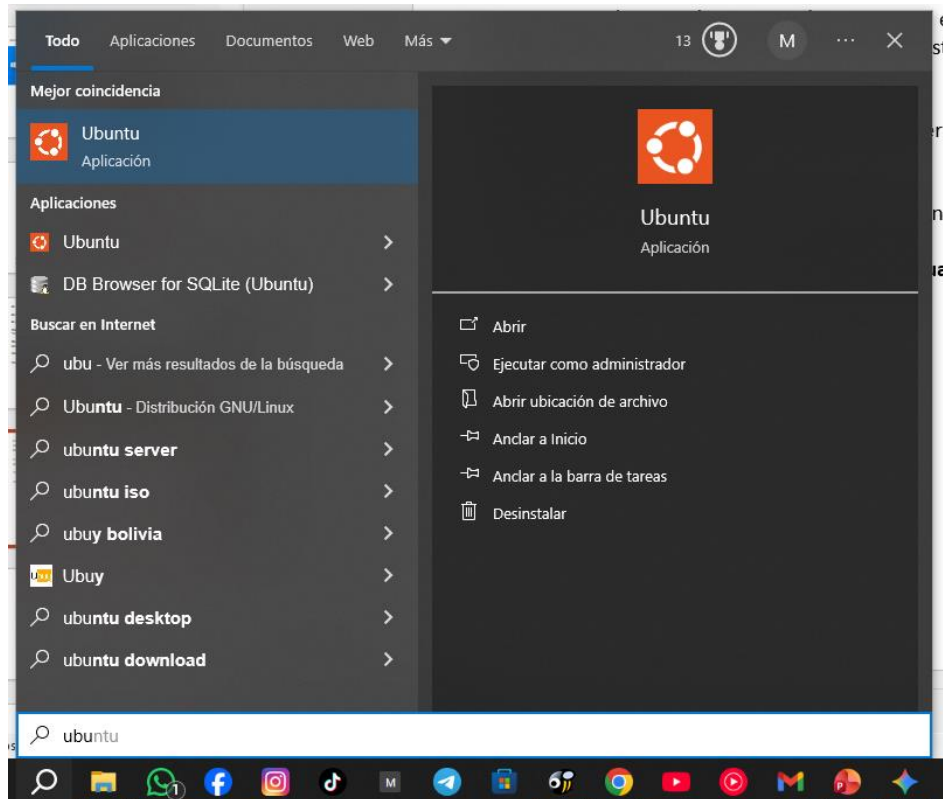
- **Habilita los componentes necesarios:** Activa las características de Windows requeridas, como "Plataforma de máquina virtual" y "Sistema de Windows para Linux".
- **Descarga el kernel de Linux más reciente:** Obtiene el paquete oficial de Microsoft para que el sistema operativo Linux pueda correr sobre Windows.
- **Instala la distribución predeterminada:** Por defecto, descarga e instala **Ubuntu** (la versión más común y amigable para principiantes).
- **Configura el entorno:** Prepara los archivos necesarios para que la integración entre Windows y Linux funcione a la perfección.



Una vez que el comando termine de ejecutarse, el sistema pedirá reiniciar la computadora. Este paso es obligatorio para que los cambios en las características del sistema se apliquen correctamente. Después del reinicio, el sistema quedará listo para usar **Linux a través de WSL**, permitiendo ejecutar una distribución de Linux dentro de Windows de forma integrada y funcional.

FORMAS PRINCIPALES DE USAR UNA DISTRIBUCIÓN DE LINUX

ABRIR UNA DISTRIBUCIÓN DE LINUX DESDE INICIO



USO POR COMANDOS (TERMINAL)

Usando el comando “wsl” podemos abrir una distribución ,pero primero debemos verificar qué distribuciones tenemos instaladas en el sistema y cuál está configurada como “**predeterminada**”. Esto es importante porque la distribución predeterminada es la que se abrirá automáticamente cuando usemos el comando **wsl** sin especificar ninguna otra.

Ejecutamos estos comandos en CMD :

`wsl -l`

Ver distros instaladas

```
C:\Windows\system32\CMD.exe
Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USUARIO>wsl -l
Distribuciones de subsistema de Windows para Linux:
Ubuntu (Predeterminado) ←
```

Abrir la distro por defecto :

`wsl`

```
joel@DESKTOP-199EVQE: ~
joel@DESKTOP-199EVQE:~$
```

Comandos más importantes de WSL

1. Descargar / instalar una distro :

```
wsl --install -d Ubuntu
```

(Podemos cambiar por otra distro como Debian, Kali, etc.)

2. Ver lista de distribuciones instaladas

```
wsl -l -v
```

Muestra:

Distros instaladas

Estado (encendida o apagada)

3 Abrir una distribución específica

```
wsl -d Ubuntu
```

Abre directamente Ubuntu (o la que indiques)

4. Cambiar la distribución predeterminada

```
wsl --set-default Ubuntu
```

Ahora Ubuntu será la que se abre con: wsl

5. Eliminar una distro en WSL

```
wsl --unregister Ubuntu
```

Ahora que ya tenemos Linux dentro de Windows usando el kernel a través de WSL, esto nos permite instalar y utilizar Docker.

PROCESO DE INSTALACIÓN DE DOCKER

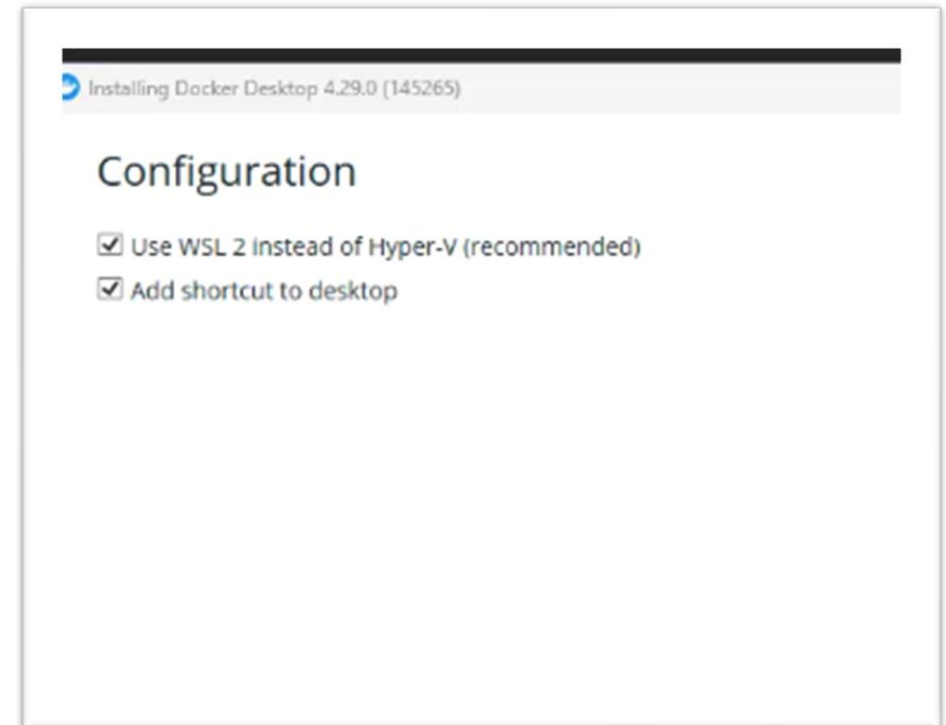
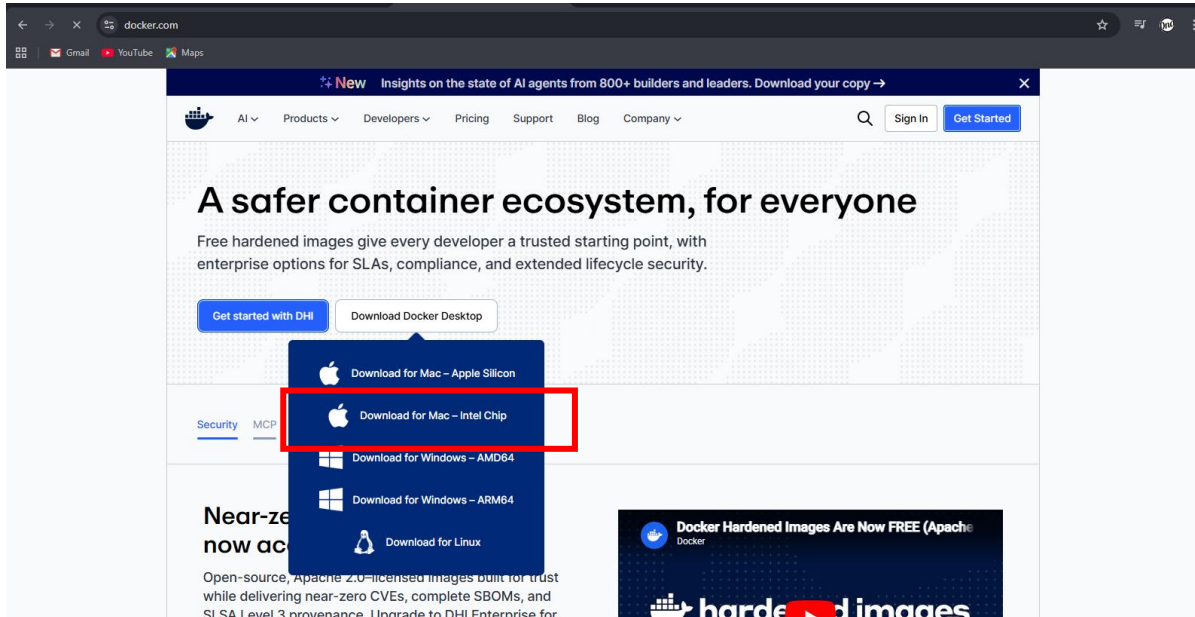
1. Descargar Docker Desktop

- Ve a la página oficial:

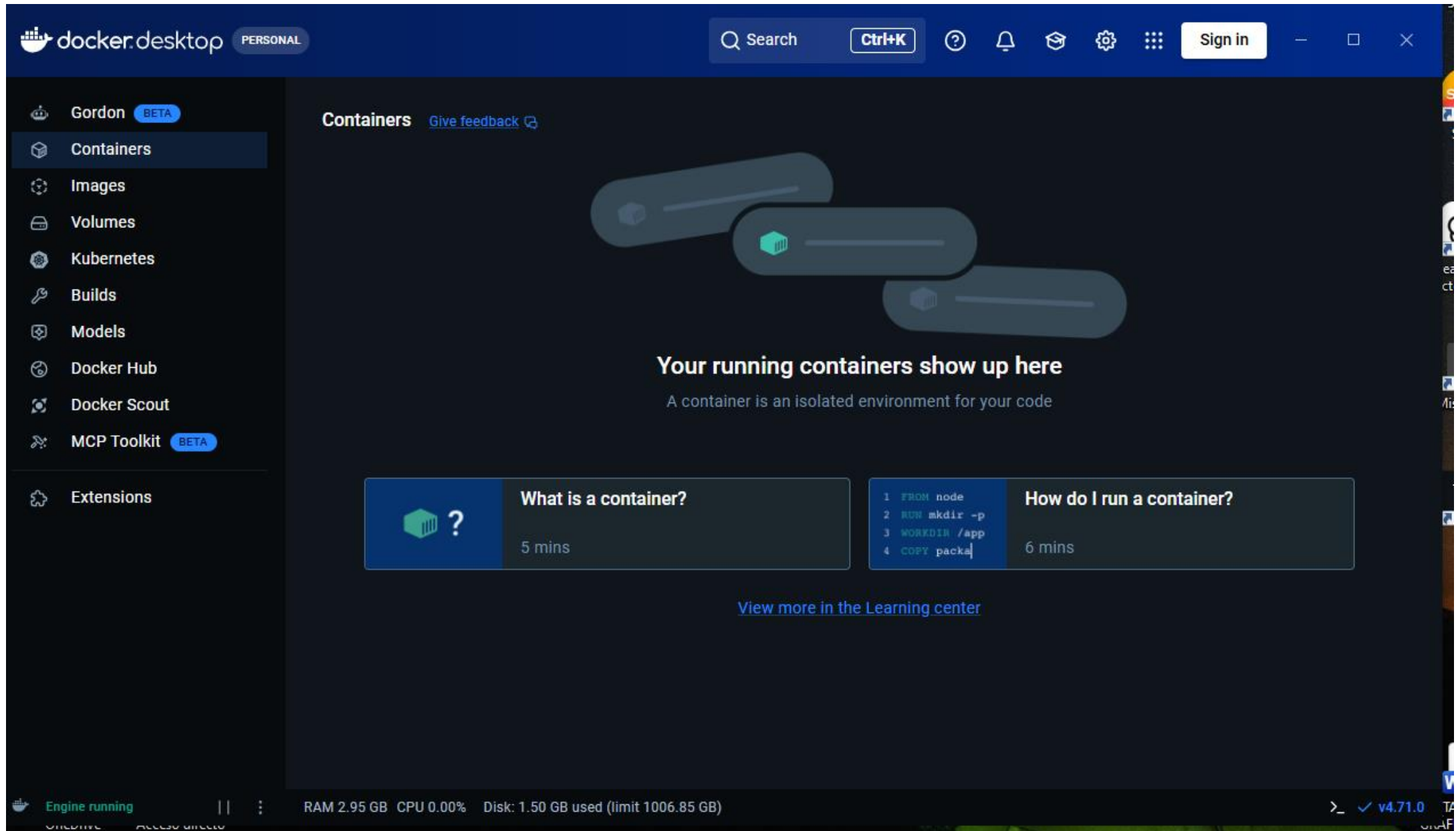
-  <https://www.docker.com/products/docker-desktop/>

- Descarga la versión para **Windows**

- Ejecutamos el instalador .exe con permisos de administrador
- Marca la opción:

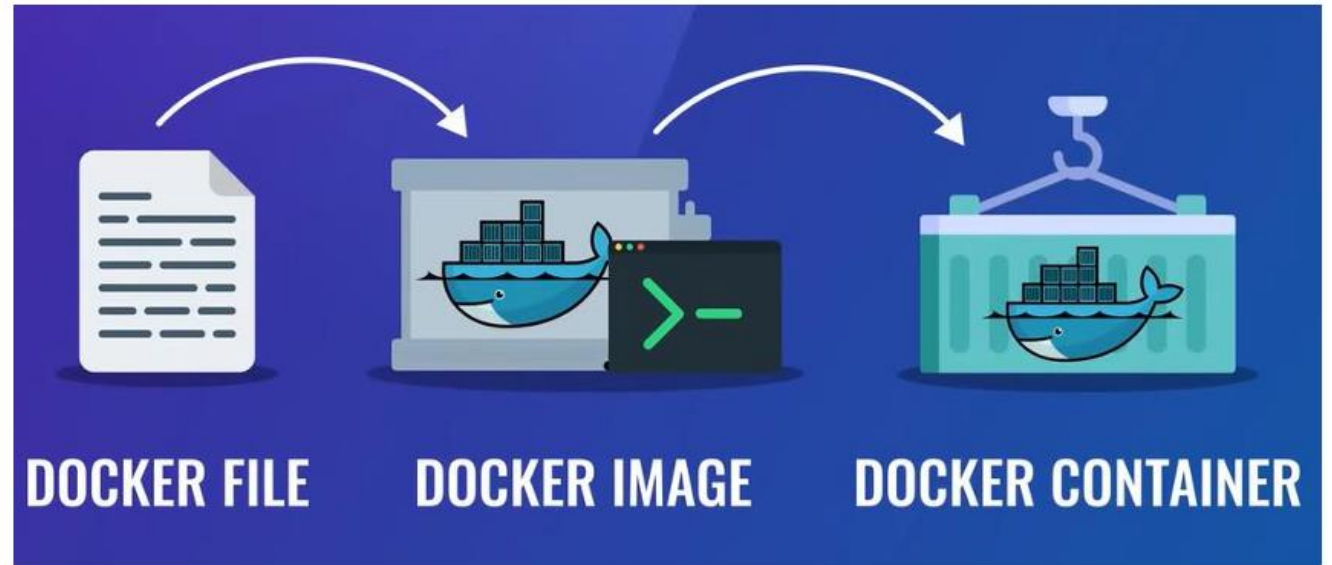


INCIO



COMPONENTES BÁSICOS DE DOCKER

- **Dockerfile:** archivo de texto que contiene una secuencia de instrucciones para construir una imagen paso a paso (por ejemplo, instalar dependencias y copiar el código fuente).
- **Imágenes de Docker:** plantillas de solo lectura que definen el estado inicial de un contenedor; incluyen el sistema de archivos y las instrucciones para ejecutar la aplicación.
- **Contenedores:** instancias ejecutables de una imagen; pueden crearse, iniciarse, detenerse y eliminarse de forma rápida. Docker automatiza la construcción de imágenes y la ejecución de contenedores, facilitando la integración continua, los despliegues reproducibles y la colaboración entre equipos de desarrollo y operaciones



En : <https://github.com/moby/moby> podemos ver el **proyecto “Moby”** que es la base de Docker, donde está el código del motor que crea y ejecuta contenedores.

Moby / Moby

Escribe para buscar

<> Código

Asuntos 3.3k

Solicitudes de extracción 542

Discusiones

Comportamiento

Proyectos

Modelos

Wiki

Seguridad y calidad 21

Perspectivas

Moby

Público

Mirar 2808

Tenedor 18.9k

Estrella 71.5k

maestro

18 sucursales

561 etiquetas

Ir al archivo

Agregar archivo

<> Código

thaJeztah

Combinar la solicitud de extracción n.º 52459 de thaJeztah/module_differ

a2d62b3 · ayer

56.382 compromisos

.devcontainer	devcontainer: Utilizar un destino devcontainer independiente	Hace 2 años
.github	gha/validate-milestone: Gestionar la actualización de version...	Hace 5 días
API	Combinar la solicitud de extracción n.º 52358 de geekcoderr/...	la semana pasada
cliente	Proveedor: github.com/moby/moby/api v1.54.2	la semana pasada
comando	chore(deps): actualizar la etiqueta docker tonistiigi/xx a v1.9.0	Hace 3 meses
contribución	Combinar la solicitud de extracción n.º 47103 de AkihiroSuda...	Hace 3 días
demonio	Combinar la solicitud de extracción n.º 52456 de AkihiroSuda...	Hace 3 días
versión de Docker	modernizar: usar strings.Builder en lugar de la concatenación...	Hace 4 meses
documentos	Actualizar RootlessKit (3.0.0)	Hace 3 días
definiciones de errores	errdefs: eliminar utilidades lsXXX obsoletas	Hace 7 meses
cortar	Combinar la solicitud de extracción n.º 52459 de thaJeztah/m...	ayer
integración-ci	Corrige errores tipográficos y ortográficos en comentarios, pr...	mes pasado
integración	integración: reducir el tamaño de los tests de integración	Hace 2 días

Acerca de

El Proyecto Moby: un proyecto colaborativo para el ecosistema de contenedores con el fin de ensamblar sistemas basados en contenedores.

[mobyproject.org/](#)

ir

estibador

golang

contenedores

Léame

Licencia Apache-2.0

Contribuyendo

Política de seguridad

Actividad

Propiedades personalizadas

71.500 estrellas

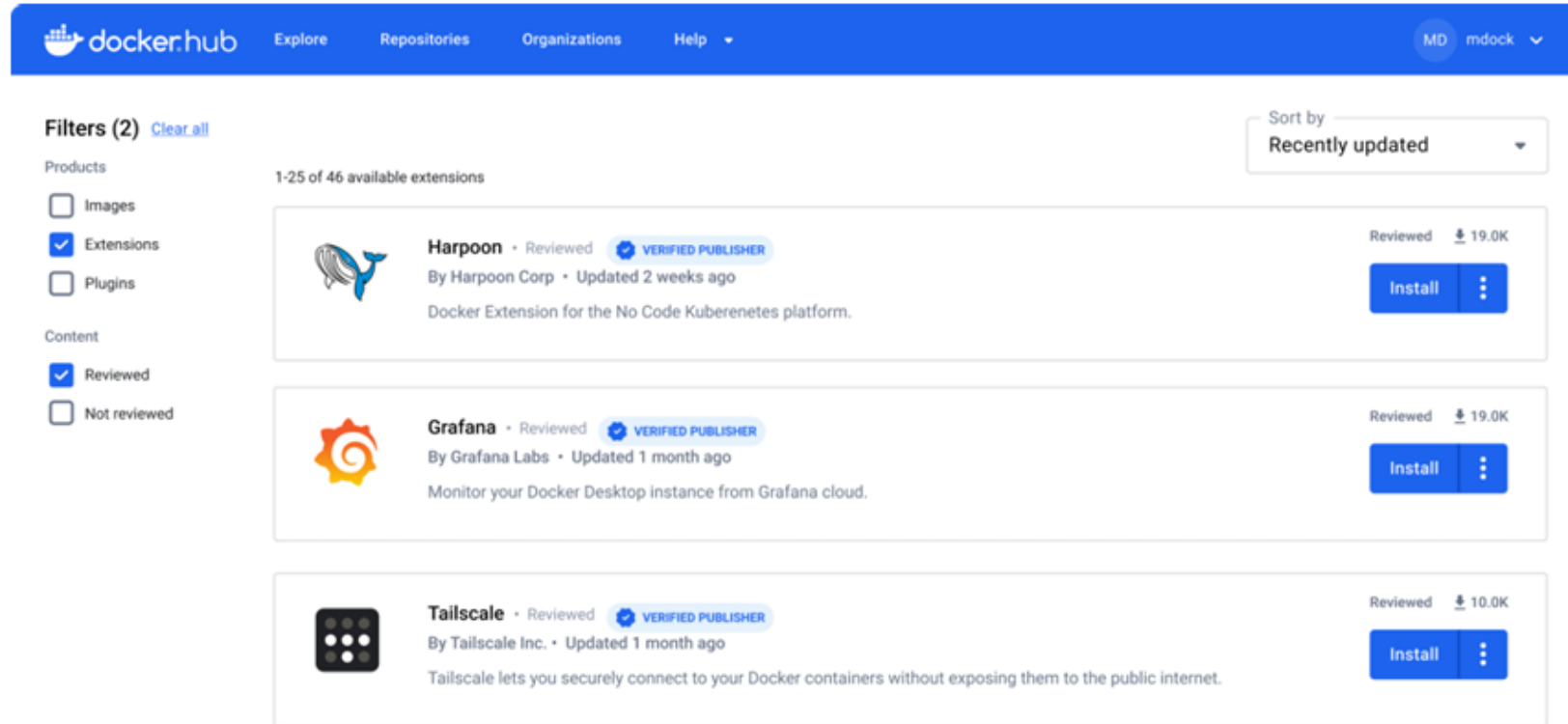
2.800 espectadores

Horquillas de 18.9k

Repositorio de informes

Comunicados 284

DOCKER HUB



The screenshot shows the Docker Hub website interface. At the top is a blue navigation bar with the Docker Hub logo and links for Explore, Repositories, Organizations, and Help. On the right of the bar, there's a user profile for 'MD' with the name 'mdock' and a dropdown arrow. Below the navigation bar, on the left, is a 'Filters (2)' section with a 'Clear all' link. It has two categories: 'Products' with checkboxes for 'Images', 'Extensions' (checked), and 'Plugins'; and 'Content' with checkboxes for 'Reviewed' (checked) and 'Not reviewed'. To the right of the filters, it says '1-25 of 46 available extensions'. Further right is a 'Sort by' dropdown menu set to 'Recently updated'. The main area displays three extension cards. Each card includes an icon, the extension name, a 'Reviewed' status with a 'VERIFIED PUBLISHER' badge, the publisher name, the update time, a download count, and an 'Install' button with a three-dot menu. The extensions are Harpoon, Grafana, and Tailscale.

Filters (2) [Clear all](#)

Products

- ☐ Images
- ☒ Extensions
- ☐ Plugins

Content

- ☒ Reviewed
- ☐ Not reviewed

1-25 of 46 available extensions

Sort by **Recently updated**

**Harpoon** • Reviewed **VERIFIED PUBLISHER**
By Harpoon Corp • Updated 2 weeks ago
Reviewed  19.0K
[Install](#) 

**Grafana** • Reviewed **VERIFIED PUBLISHER**
By Grafana Labs • Updated 1 month ago
Reviewed  19.0K
[Install](#) 

**Tailscale** • Reviewed **VERIFIED PUBLISHER**
By Tailscale Inc. • Updated 1 month ago
Reviewed  10.0K
[Install](#) 

Docker Extension for the No Code Kubernetes platform.

Monitor your Docker Desktop instance from Grafana cloud.

Tailscale lets you securely connect to your Docker containers without exposing them to the public internet.

Docker Hub es el registro oficial y la "nube" donde se almacenan y comparten imágenes de contenedores, funcionando esencialmente como una biblioteca o tienda de aplicaciones (tipo App Store) para desarrolladores de sistemas. Su utilidad principal radica en permitirte buscar, descargar y desplegar imágenes preconfiguradas y oficiales de software —como servidores web, bases de datos o entornos de programación—, lo que elimina la necesidad de configurar todo desde cero, facilita la estandarización de tus proyectos y garantiza que, al usar versiones etiquetadas (tags), tu aplicación funcione exactamente igual en cualquier máquina donde se ejecute.



Comandos Básicos de Docker

Imágenes

```
docker pull nombre_imagen
```

Descarga una imagen

```
docker build -t nombre .
```

Crea una imagen

```
docker images
```

Lista imágenes

Contenedores

```
docker run nombre_imagen
```

Ejecuta contenedor

```
docker ps
```

Contenedores activos

```
docker ps -a
```

Todos los contenedores

Gestión

```
docker start ID
```

Inicia contenedor detenido

```
docker stop ID
```

Detiene contenedor

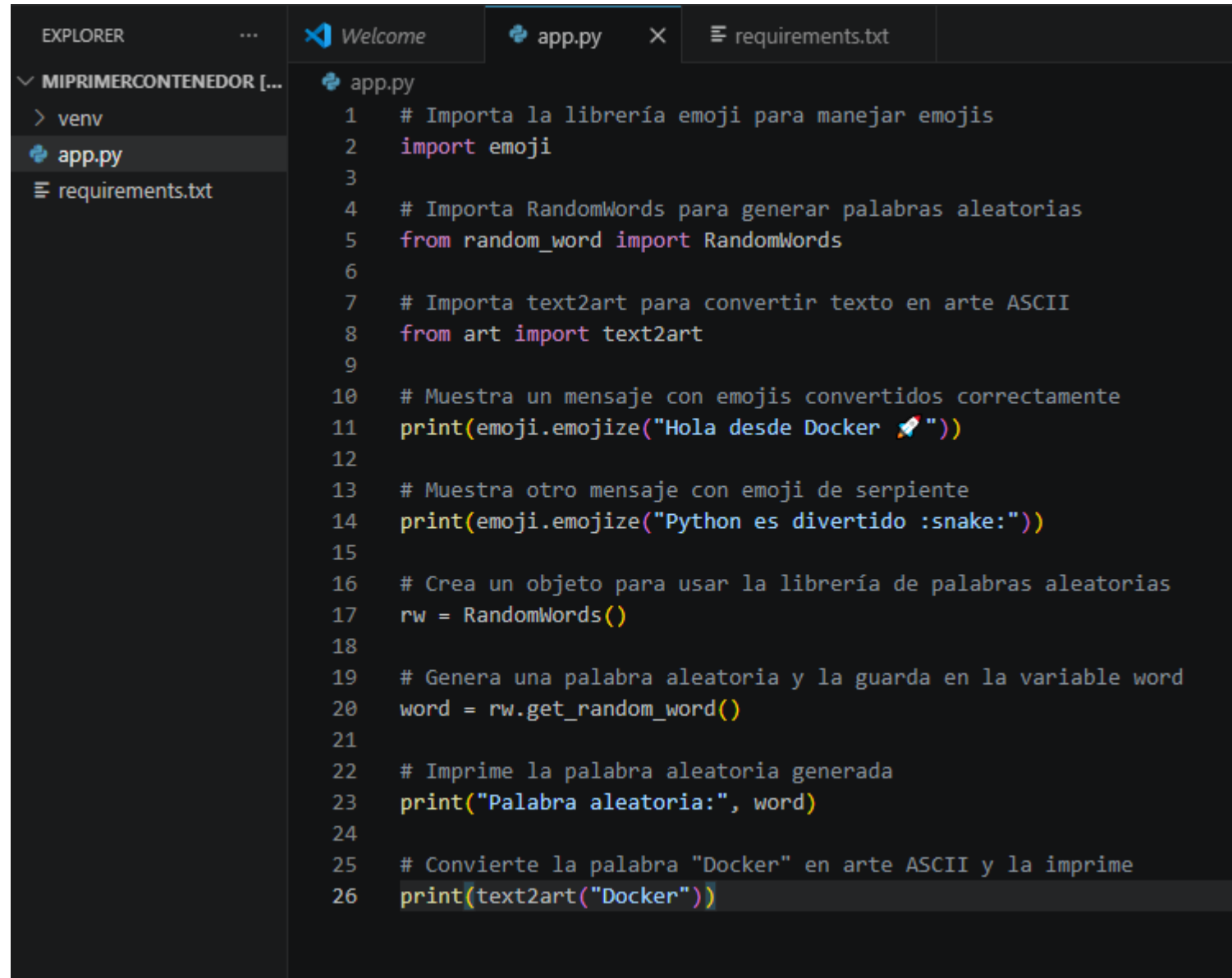
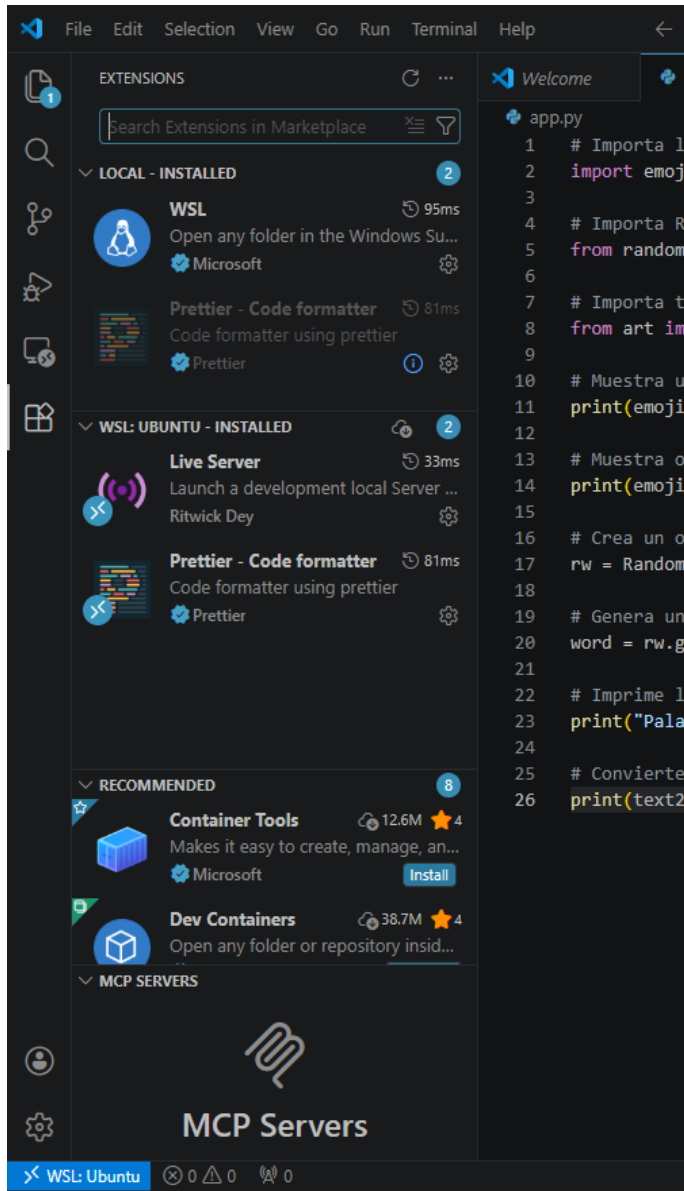
```
docker rm ID
```

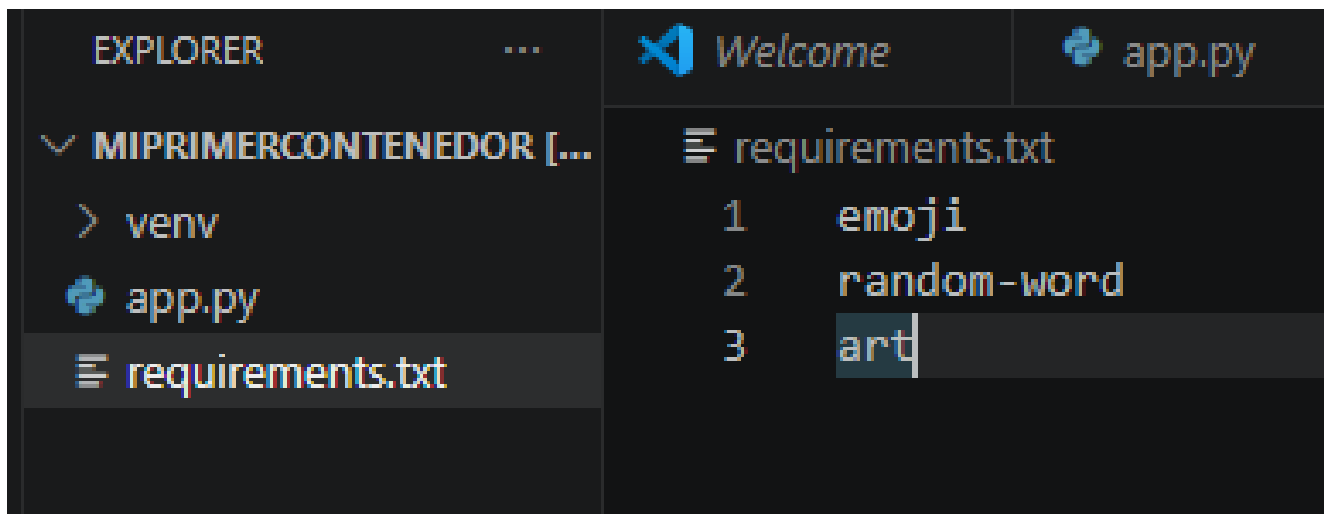
Elimina contenedor

```
docker rename actual nuevo
```

Renombra contenedor

MI PRIMER CONTENEDOR





```
● joel@DESKTOP-199EVQE:~/MiPrimerContenedor$ source venv/bin/activate
```

```
• (venv) joel@DESKTOP-199EVQE:~/MiPrimerContenedor$ python app.py
```

Hola desde Docker 🚀

Python es divertido 🐍

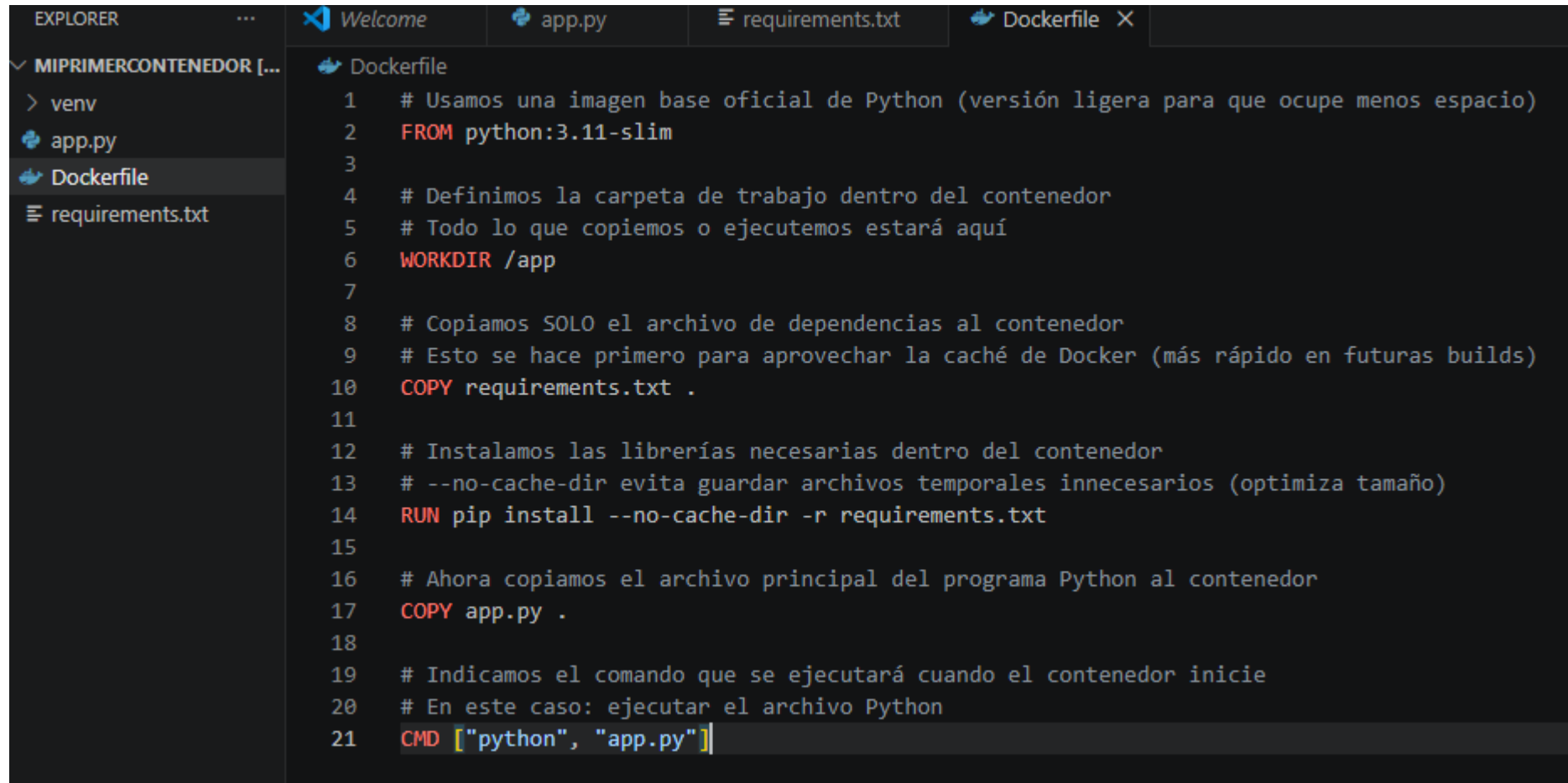
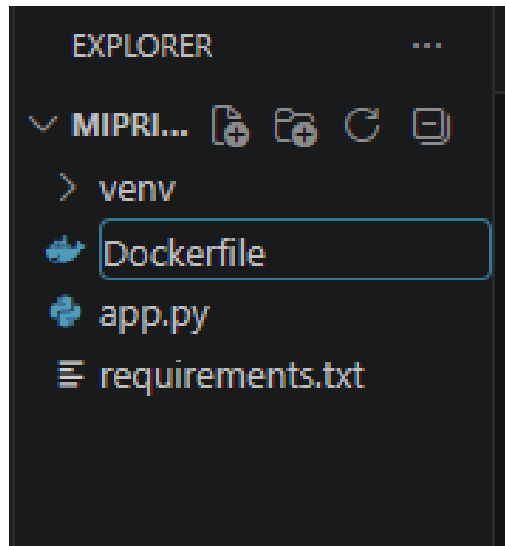
Palabra aleatoria: threnode

Deleuze

```

○ (venv) joel@DESKTOP-199EVQE:~/MiPrimerContenedor$

```

```
EXPLORER Welcome app.py requirements.txt Dockerfile X
▼ MIPRIMERCONTENEDOR [...]
  > venv
  app.py
  Dockerfile
  requirements.txt

Dockerfile
1  # Usamos una imagen base oficial de Python (versión ligera para que ocupe menos espacio)
2  FROM python:3.11-slim
3
4  # Definimos la carpeta de trabajo dentro del contenedor
5  # Todo lo que copiamos o ejecutemos estará aquí
6  WORKDIR /app
7
8  # Copiamos SOLO el archivo de dependencias al contenedor
9  # Esto se hace primero para aprovechar la caché de Docker (más rápido en futuras builds)
10 COPY requirements.txt .
11
12 # Instalamos las librerías necesarias dentro del contenedor
13 # --no-cache-dir evita guardar archivos temporales innecesarios (optimiza tamaño)
14 RUN pip install --no-cache-dir -r requirements.txt
15
16 # Ahora copiamos el archivo principal del programa Python al contenedor
17 COPY app.py .
18
19 # Indicamos el comando que se ejecutará cuando el contenedor inicie
20 # En este caso: ejecutar el archivo Python
21 CMD ["python", "app.py"]
```

```
docker build
```

construir imagen

```
-t
```

darle nombre (tag)

```
mi-python-app
```

nombre de la imagen

```
.
```

carpeta actual donde está el Dockerfile

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
joel@DESKTOP-199EVQE:~/MiPrimerContenedor$ docker build -t mi-python-app .  
[+] Building 34.4s (10/10) FINISHED  
=> [internal] load build definition from Dockerfile  
=> => transferring dockerfile: 865B  
=> [internal] load metadata for docker.io/library/python:3.11-slim  
=> [internal] load .dockerignore  
=> => transferring context: 2B  
=> [1/5] FROM docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d7607  
=> => resolve docker.io/library/python:3.11-slim@sha256:6d85378d88a19cd4d7607  
=> => sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d765d7c799d46035ceb563  
=> => sha256:f3ba2250c52436e41e9f2f3830af92de466fcacc2d848264e1f9fdc76975d803  
=> => sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e13f94fa2f12ae4b221aae996b9  
=> => extracting sha256:91ff8760033c3b9bc9c3a3bcc6ff66c113ec9e13f94fa2f12ae4b  
=> => extracting sha256:f3ba2250c52436e41e9f2f3830af92de466fcacc2d848264e1f9f  
=> => extracting sha256:7ccd73948dde4b7f1623ac6bf9b58706fb83d0a363d765d7c799d  
=> [internal] load build context
```

 Gordon BETA Containers Images Volumes Kubernetes Builds Models Docker Hub Docker Scout MCP Toolkit BETA ExtensionsImages [Give feedback](#)

Local

My Hub

1.37 GB / 0 Bytes in use 3 images

Last refresh: 23 hours ago  Search

☐		Name	Tag	Image ID	Created	Size	Actions
☐	●	nginx	latest	6e23479198b9	8 days ago	240.02 MB	   
☐	●	mi-backend	latest	53acacfa4913	5 hours ago	1.62 GB	   
☐	○	mi-python-app	latest	a32f26a36b57	2 minutes ago	253.94 MB	   

Walkthroughs

```
1 FROM node
2 RUN mkdir -p
3 WORKDIR /app
4 COPY packa
```

How do I run a container?

6 mins



Run Docker Hub images

5 mins

[View more in the Learning center](#)

C:\Windows\system32\CMD.exe

Microsoft Windows [Versión 10.0.19045.6466]

(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USUARIO>docker create mi-python-app

d5b9e4e2cb674bcf3ab60bb98ce51c22ff99a87d8b54030a8a462f9cd2d2e0cc

C:\Users\USUARIO>_

 Gordon BETA

 Containers

 Images

 Volumes

 Kubernetes

 Builds

 Models

 Docker Hub

 Docker Scout

 MCP Toolkit BETA

 Extensions

Containers [Give feedback](#)

Container CPU usage 

No containers are running.

Container memory usage 

No containers are running.

[Show charts](#)

 Search



Only show running containers

	Name	Container ID	Image	Port(s)	CPU (%)	Last started	Actions
	sleepy_blackwe	d5b9e4e2cb67	mi-python-i		N/A		   

C:\Windows\system32\CMD.exe

Microsoft Windows [Versión 10.0.19045.6466]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\USUARIO>docker ps -a

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS	NAMES
79a9c99e9845	mi-python-app	"python app.py"	About a minute ago	Created		boring_shamir

C:\Users\USUARIO>docker rename boring_shamir mi-python-app

C:\Users\USUARIO>docker start -a mi-python-app

Hola desde Docker

Python es divertido

Palabra aleatoria: taistril

Docker

C:\Users\USUARIO>

Gordon BETA

Containers

Images

Volumes

Kubernetes

Builds

Models

Docker Hub

Docker Scout

MCP Toolkit BETA

Extensions

Containers [Give feedback](#)

Container CPU usage ⓘ

No containers are running.

Container memory usage ⓘ

No containers are running.

[Show charts](#)

Search



Only show running containers

☐		Name	Container ID	Image	Port(s)	CPU (%)	Last	Actions
☐	○	mi-python-app	79a9c99e9845	mi-python:		N/A	in 11	

Showing 1 item



Engine running



RAM 1.79 GB CPU 0.25% Disk: 3.43 GB used (limit 1006.85 GB)



v4.71.0

 Gordon BETA

 Containers

 Images

 Volumes

 Kubernetes

 Builds

 Models

 Docker Hub

 Docker Scout

 MCP Toolkit BETA

 Extensions

[Containers](#) / mi-python-app



mi-python-app

 79a9c99e9845 

 [mi-python-app:latest](#)

STATUS

Exited (0) (in 10 seconds)



Logs

Inspect

Bind mounts

Exec

Files

Stats

Hola desde Docker 🚀

Python es divertido 🐍

Palabra aleatoria: taistril

```

  _ _ _ _ _
 | _ \ _ _ _ | | _ _ _
 | | | / _ \ / _ | | / _ \
 | | | ( ) | ( _ | < | _ /
 | _ / _ \ _ \ | | \ _ \

```

PRUEBA : SERVIDOR WEB NGINX

Paso 1: Ejecutar tu primer contenedor

Abre tu terminal (PowerShell, CMD, o la terminal de VS Code) y ejecuta el siguiente comando:

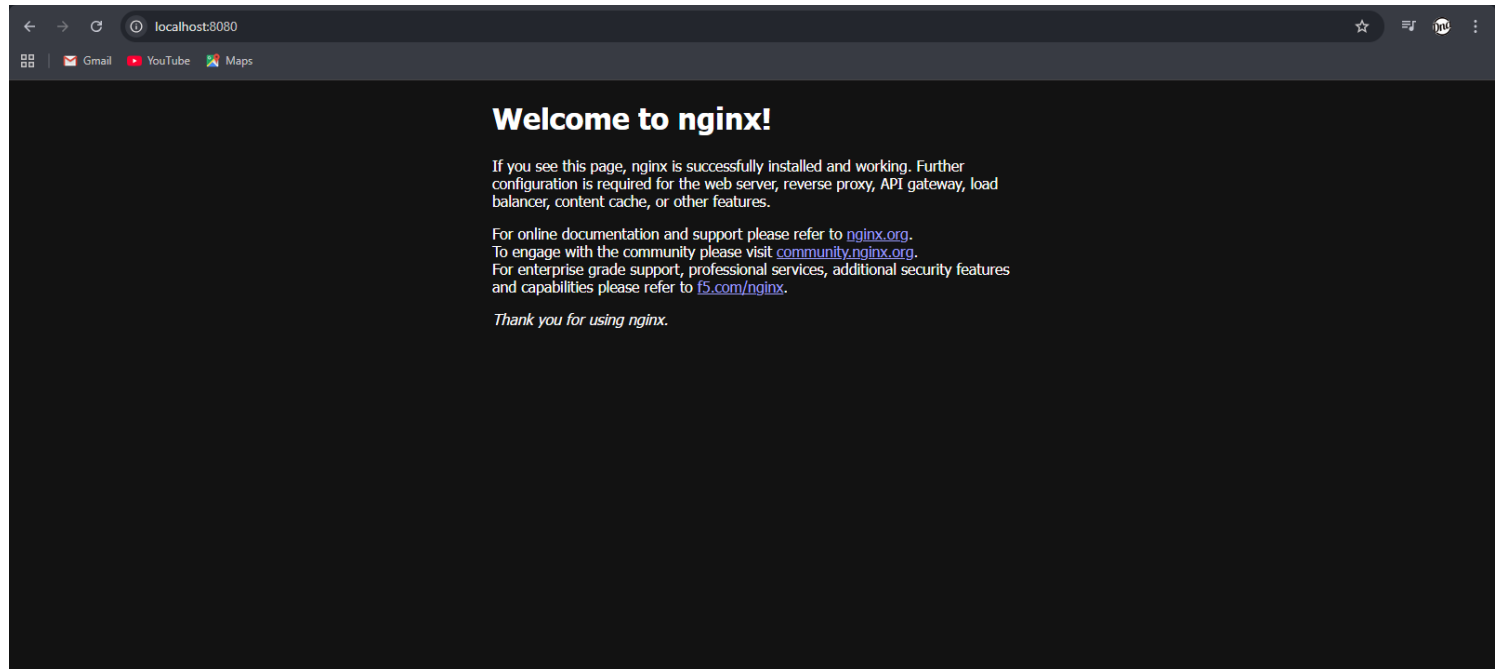
```
docker run -d -p 8080:80 --name mi-servidor-web nginx
```

¿Qué está haciendo este comando?

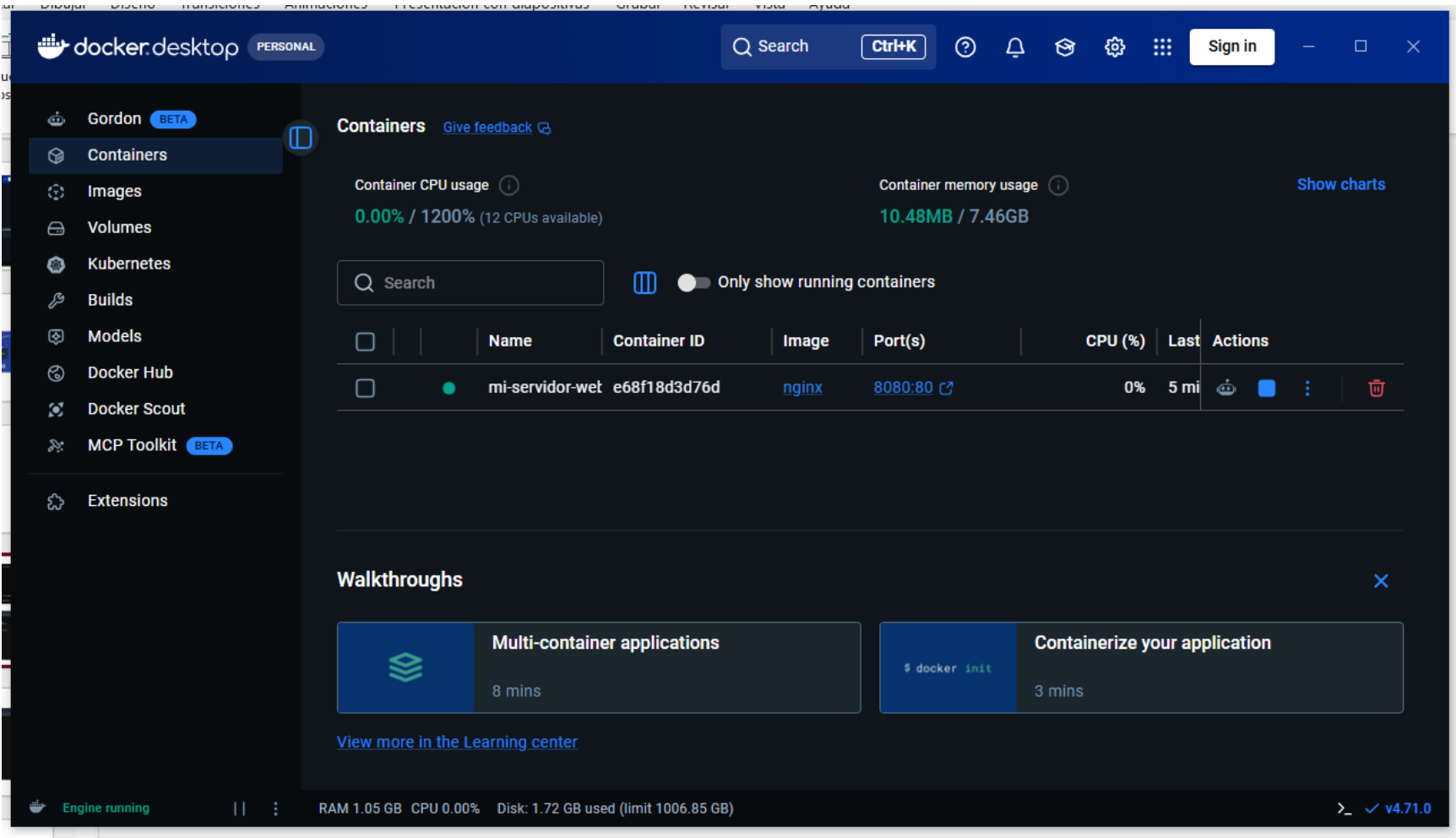
- `docker run` : Le dice a Docker que cree y ejecute un contenedor a partir de una imagen.
- `-d` (Detached): Ejecuta el contenedor en segundo plano, así tu terminal queda libre.
- `-p 8080:80` : Esto es el **mapeo de puertos**. Conecta el puerto `8080` de tu computadora al puerto `80` (donde escucha Nginx) dentro del contenedor.
- `--name mi-servidor-web` : Le asigna un nombre amigable al contenedor.
- `nginx` : Es la imagen base que Docker descargará automáticamente desde Docker Hub.

```
C:\Users\USUARIO>docker run -d -p 8080:80 --name mi-servidor-web nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
801a1ad15b4e: Pull complete
4677c2a9a3d4: Pull complete
ff048f1f2159: Pull complete
85c66128325a: Pull complete
3531af2bc2a9: Pull complete
ce776bbcdad: Pull complete
677c63196868: Pull complete
54c38c75806e: Download complete
69989ccd189b: Download complete
Digest: sha256:6e23479198b998e5e25921dff8455837c7636a67111a04a635cf1bb363d199dc
Status: Downloaded newer image for nginx:latest
e68f18d3d76d9c285a056cdc42f76289f69de13114f4e51526669496d6f5fd6c
```

- En el navegador: Abrimos **http://localhost:8080**. Deberías ver una página que dice "Welcome to nginx!".



Ahora regresamos a docker desktop. Veremos que la lista ya no está vacía: aparecerá mi-servidor-web con un punto verde indicando que está en ejecución.



INTRODUCCION A LA ORQUESTACION

Una aplicación puede construirse como un sistema monolítico (todo en un solo bloque) o mediante una arquitectura de microservicios, donde la aplicación se divide en múltiples partes independientes.



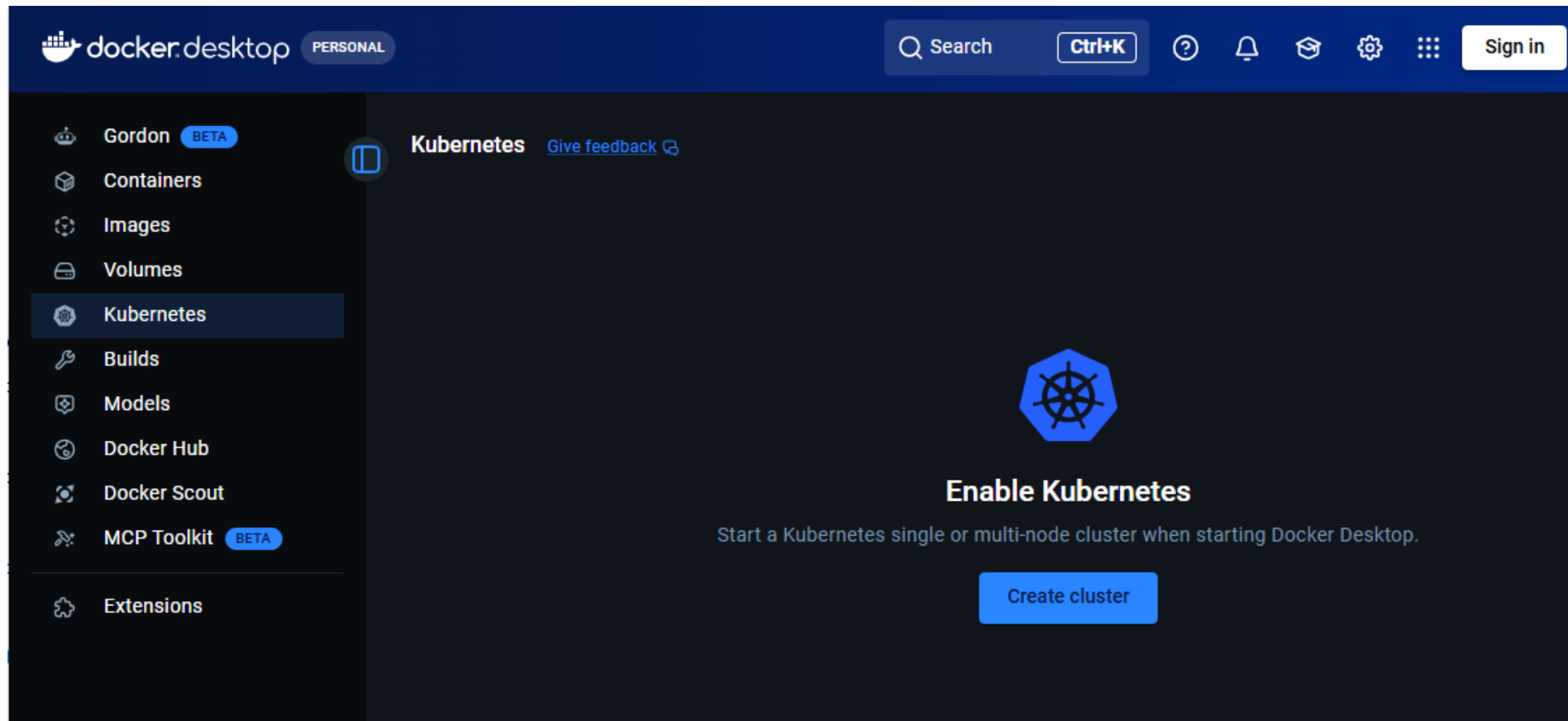
Debemos entender que una aplicación moderna no es una sola pieza, sino un conjunto de servicios que trabajan juntos; por ejemplo, una aplicación de comercio electrónico puede tener un servicio para usuarios, otro para productos, otro para pagos y otro para pedidos. Cada uno de estos servicios puede ejecutarse en su propio contenedor, lo que facilita su desarrollo, mantenimiento y escalabilidad. Sin embargo, cuando se tienen muchos contenedores funcionando al mismo tiempo, surge la necesidad de gestionarlos de manera eficiente. Aquí es donde entra la orquestación de contenedores, utilizando herramientas como Kubernetes, que permite organizar, coordinar y automatizar el funcionamiento de todos estos servicios, asegurando que la aplicación funcione correctamente como un todo. La orquestación de contenedores es el proceso automatizado de despliegue, escalado, monitoreo y recuperación de contenedores en un entorno distribuido.



KUBERNETES

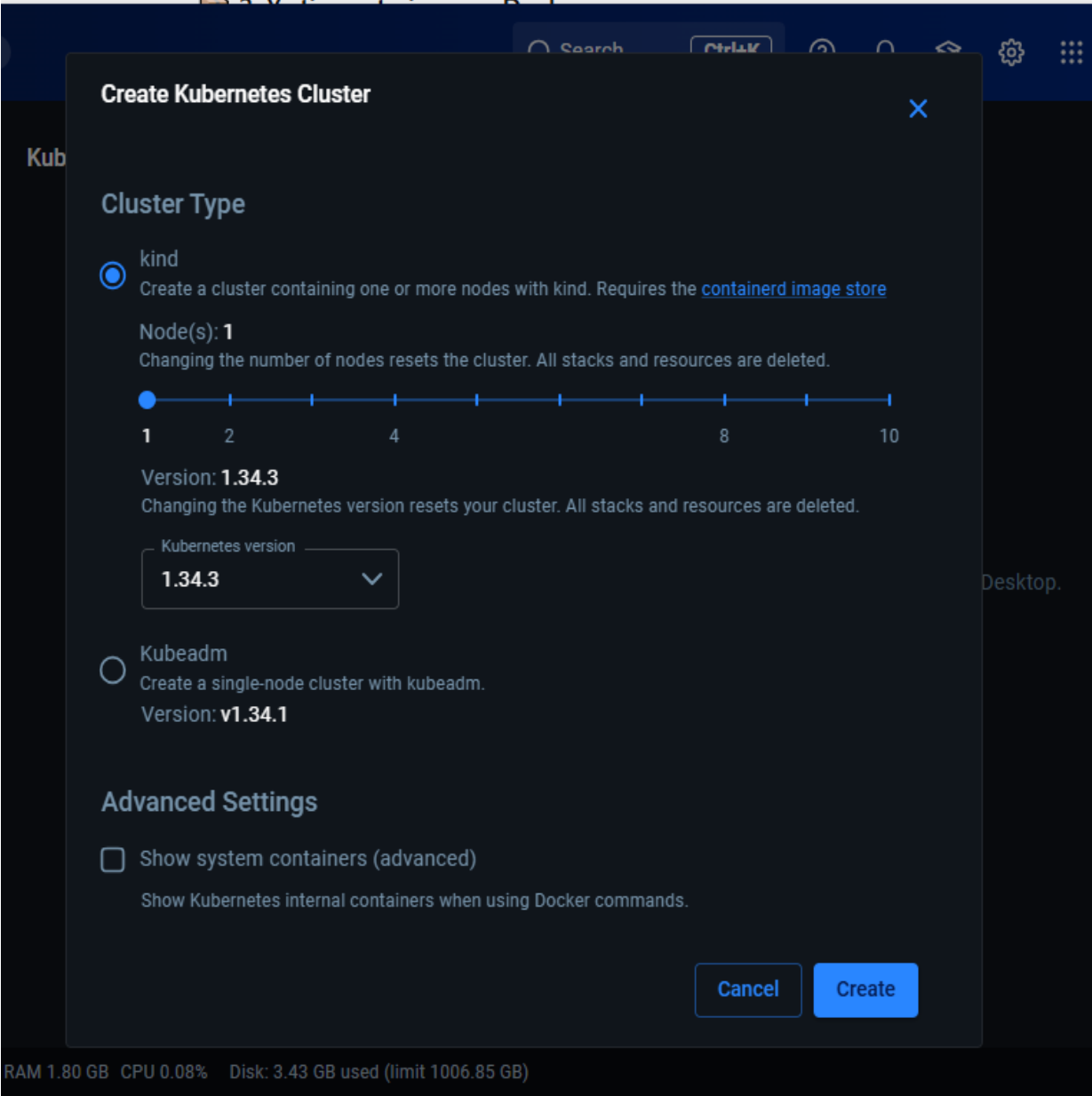


Kubernetes es una herramienta que se utiliza para administrar y coordinar muchos contenedores (como los de Docker) de forma automática, especialmente cuando una aplicación es grande y está dividida en varios servicios; en lugar de ejecutar y controlar manualmente cada contenedor, Kubernetes se encarga de iniciarlos, reiniciarlos si fallan, distribuir la carga entre ellos y escalar la aplicación si hay más usuarios, todo a través de configuraciones que se definen en archivos, lo que lo hace muy usado en entornos de producción y empresas grandes donde se necesita que los sistemas funcionen de manera estable, automática y sin intervención constante del desarrollador.



Un **cluster** es un conjunto de varias computadoras que trabajan juntas como si fueran una sola máquina para realizar tareas de forma más rápida, estable y eficiente; en lugar de depender de un solo equipo, el trabajo se reparte entre varios, lo que permite soportar más carga, mejorar el rendimiento y evitar que el sistema se detenga si una de las computadoras falla.

Componente	Qué es	Función
Control Plane	Cerebro del cluster	Decide qué se ejecuta, dónde y cuándo. Administra todo el sistema.
API Server	Punto de entrada	Recibe las órdenes (kubectl, apps) y las envía al cluster.
Scheduler	Programador	Decide en qué máquina se ejecuta cada contenedor.
Controller Manager	Controlador	Vigila que todo esté funcionando como se definió (repara fallos).
etcd	Base de datos interna	Guarda toda la información del cluster (estado, configuraciones).
Worker Node	Máquina de trabajo	Ejecuta los contenedores de las aplicaciones.
Kubelet	Agente del nodo	Se asegura de que los contenedores se ejecuten correctamente.
Container Runtime	Motor de contenedores	Ejecuta los contenedores (Docker, containerd).
Pod	Unidad mínima	Grupo de uno o más contenedores que se ejecutan juntos.



Operación	Sub-opción / Parámetro	Descripción Funcional
Cluster Type	kind	Crea un clúster donde cada nodo es un contenedor de Docker. Ideal para desarrollo rápido y ligero.
	Node(s)	Control deslizante para definir la cantidad de nodos (1 a 10). Más nodos permiten simular escenarios de alta disponibilidad.
	Version (kind)	Permite seleccionar la versión específica de Kubernetes (ej. 1.34.3) para asegurar compatibilidad con el código.
	Kubeadm	Utiliza la herramienta estándar de la industria para configurar un clúster de un solo nodo, más cercano a un entorno real.
Advanced Settings	Show system containers	Al activarse, permite que los contenedores internos de Kubernetes sean visibles mediante comandos de Docker.
Acciones	Create	Inicia el despliegue del clúster. Advertencia: Cambiar la versión o los nodos después de creado borrará todos los datos previos.
	Cancel	Aborta la configuración y cierra la ventana sin realizar cambios en el sistema.